

Day 13 - Notes

Salesforce DevOps, CI/CD & Enterprise Deployment Workflow Notes

Goal of Day 13

Understand how real Salesforce applications are:

- Deployed safely
- Managed by teams
- Tested before release
- Delivered using CI/CD pipelines
- Maintained in enterprise environments

This day focused on:

- DevOps thinking
 - Deployment workflow
 - GitHub collaboration
 - CI/CD concepts
 - Enterprise release management
-

1. Org Development Model

What is an Org Development Model?

An Org Development Model defines how Salesforce teams organize:

- Development
- Testing
- Validation
- Deployment
- Production release

Enterprise systems separate environments to reduce risks.

Common Salesforce Environments

Developer Sandbox

Used by individual developers for coding and testing.

Developer Pro Sandbox

Used for advanced development and integration testing.

Partial Copy Sandbox

Contains partial production data for testing workflows.

Full Sandbox

Complete copy of production used for staging and final testing.

Production Org

Live environment used by real users.

Why Environment Separation is Important

Benefits:

- Prevents production failures
 - Protects business data
 - Enables safer testing
 - Supports rollback planning
 - Reduces deployment risk
 - Improves reliability
-

Typical Development Lifecycle

Step 1 — Development

Developers create features inside sandboxes.

Step 2 — Source Control

Changes are committed to GitHub.

Step 3 — Testing

Automated and manual testing validate the code.

Step 4 — Validation

Deployment packages are validated.

Step 5 — Staging

Features are tested in staging environments.

Step 6 — Production Deployment

Approved changes are deployed to production.

2. Salesforce DevOps

What is DevOps?

DevOps is a software engineering approach that combines:

- Development
- Testing
- Deployment
- Monitoring
- Operations

Main goals:

- Faster delivery
- Better collaboration
- Reliable deployments

- Automation
 - Stability
-

Why DevOps is Important

Enterprise Salesforce systems support:

- Thousands of users
- Critical business operations
- Large datasets
- Complex automation
- Multiple integrations

Because of this, deployments must be managed carefully.

3. Deployment Workflow

Typical Deployment Pipeline

Developer Writes Code

↓

GitHub Commit

↓

Automated Testing

↓

Validation

↓

Staging Deployment

↓

Production Release

Benefits of Deployment Workflow

- Reduces human errors

- Improves release quality
 - Detects bugs early
 - Supports automation
 - Enables safer deployment
 - Improves collaboration
-

4. CI/CD Concepts

What is CI/CD?

CI/CD stands for:

Continuous Integration (CI)

Developers continuously merge code into a shared repository where automated testing validates changes.

Continuous Delivery / Deployment (CD)

Validated code is automatically prepared or deployed to production.

Continuous Integration (CI)

Focuses on:

- Frequent code integration
 - Automated testing
 - Early bug detection
 - Stable codebase maintenance
-

Continuous Delivery (CD)

Focuses on:

- Automated release preparation

- Deployment automation
 - Faster software delivery
 - Safer production deployment
-

Why CI/CD is Important

Without CI/CD:

- Bugs reach production easily
 - Deployments become risky
 - Releases become slow
 - Manual mistakes increase
 - Team coordination becomes difficult
-

5. GitHub Actions

What are GitHub Actions?

GitHub Actions automate workflows directly inside GitHub.

Used for:

- Automated testing
 - Validation
 - Deployment
 - CI/CD pipelines
 - Notifications
 - Build automation
-

Benefits of GitHub Actions

- Reduces manual work
- Speeds up deployment
- Improves consistency

- Supports automation
 - Improves productivity
-

Example Workflow

Developer Pushes Code

↓

GitHub Actions Trigger

↓

Automated Tests Run

↓

Validation Checks

↓

Deployment Starts

6. Enterprise Deployment Risks

Why Direct Production Editing is Dangerous

Suppose a college management system is used by:

- 50,000 students
- 500 faculty members
- Multiple administrators

Direct production changes can create serious problems.

Possible Risks

Production Bugs

Small mistakes can break workflows.

System Downtime

Users may lose access to services.

Data Loss

Incorrect logic can damage records.

Broken Automation

Flows and triggers may fail.

Security Issues

Sensitive data may become exposed.

Large Business Impact

Thousands of users may be affected.

Why Enterprise Teams Avoid Direct Production Changes

Enterprise teams prioritize:

- Stability
- Reliability
- Safe deployment
- Validation
- Rollback capability

Production systems must remain stable.

7. Team Collaboration Problems

Scenario: 10 Developers Working Together

Without GitHub, branches, testing, and deployment workflow, many issues occur.

Problems Without Version Control

Code Overwriting

Developers replace each other's work.

Merge Conflicts

Different versions create conflicts.

Lost Changes

Important work may disappear.

No Change Tracking

Teams cannot identify modifications.

Deployment Confusion

Unstable code may reach production.

Difficult Rollback

Recovering stable versions becomes difficult.

Why Branches are Important

Common branches:

- Main branch
- Development branch
- Feature branch
- Release branch
- Hotfix branch

Benefits:

- Parallel development
 - Safer experimentation
 - Controlled releases
 - Easier collaboration
-

8. Testing Before Deployment

Why Testing is Important

Testing ensures that new code does not break existing functionality.

Types of Testing

Unit Testing

Tests individual components.

Integration Testing

Tests system interactions.

Regression Testing

Ensures older features still work.

User Acceptance Testing

Validates business requirements.

Benefits of Testing

- Detects bugs early
 - Improves software quality
 - Reduces production failures
 - Supports safe deployment
-

9. Rollback and Release Management

What is Rollback?

Rollback means restoring a previous stable version after deployment failure.

Why Rollback is Important

Rollback helps:

- Recover systems quickly
 - Reduce downtime
 - Protect business operations
 - Maintain data integrity
-

Release Management

Release management controls:

- Planning releases
 - Testing releases
 - Scheduling deployment
 - Monitoring deployment
 - Handling rollback
-

10. Agentforce DX Overview

What is Agentforce DX?

Agentforce DX extends Salesforce DX for AI-enabled workflows.

Supports:

- AI agent development
 - Source-driven workflow
 - GitHub integration
 - CLI-based deployment
 - Enterprise DevOps workflow
-

Importance of Agentforce DX

Combines:

- Automation
 - Collaboration
 - Version control
 - Source-driven development
 - Enterprise deployment practices
-

11. CI/CD Workflow Explanation

Complete Workflow

Developer Writes Code



GitHub Commit



Automated Testing



Validation



Deployment



Production Release

Importance of Each Step

Step	Purpose
Development	Creates features
GitHub Commit	Tracks changes
Automated Testing	Detects bugs
Validation	Prevents unsafe deployment
Deployment	Moves code safely

Step	Purpose
Production Release	Delivers stable software

12. Enterprise Software Engineering vs Writing Code

Writing Code

Usually involves:

- Small projects
- Single developer
- Minimal testing
- Simple deployment

Enterprise Software Engineering

Requires:

- Team collaboration
- CI/CD pipelines
- Rollback planning
- Secure deployment
- Reliability engineering
- Version control
- Long-term maintenance

Major Difference

Writing code creates functionality.

Enterprise software engineering ensures functionality is:

- Reliable
- Scalable

- Secure
 - Maintainable
 - Stable for large users
-

Key Concepts Learned

- Enterprise systems require structured deployment workflow
 - CI/CD improves software reliability
 - GitHub supports collaboration and version control
 - Automated testing reduces deployment risk
 - Production systems require stability
 - Rollback capability is critical
 - DevOps combines development and operations
 - GitHub Actions automate deployment pipelines
 - Enterprise engineering is more than writing code
-

Final Outcome

Learned enterprise Salesforce deployment workflow, DevOps concepts, CI/CD pipelines, GitHub Actions, release management, rollback strategies, testing workflow, and professional software delivery lifecycle used by real engineering teams.